Mês 7 — Expansão: SVM/RF, db8/sym8, L=5 (mar/2027)

Bolsista PIBITI

Março 2027

Sumário

1 O que o plano exige **3**

2 SVM (Support Vector Machine) **4**
2.1 Importando bibliotecas 4
2.2 Criando um Pipeline Básico 4
2.3 Treinamento Simples 4
2.4 Predição e Acurácia 4
2.5 GridSearchCV para Hiperparâmetros 5
2.6 Executando o Grid Search 5

3 Random Forest **6**
3.1 Importando Random Forest 6
3.2 Instanciando o Modelo 6
3.3 Treinamento RF 6
3.4 Avaliando RF 6
3.5 Importância das Features 6

4 Novas Wavelets: db8 e sym8 **7**
4.1 Listando Wavelets 7
4.2 Propriedades db8 7
4.3 Propriedades sym8 7
4.4 Reconstrução com db8 7
4.5 Erro de reconstrução 7

5 Nível L=5 **8**
5.1 WPT Nível 5 8
5.2 Listando nós Nível 5 8
5.3 Extraindo energia L=5 8
5.4 Extração completa L=5 8
5.5 Teste L=5 vs L=4 9

6 Matriz de configuração **10**
6.1 Importando itertools 10
6.2 Definindo opções 10
6.3 Gerando combinações 10
6.4 Loop principal 10
6.5 Função get_classifier 10

7 Registrar experimentos **12**
7.1 Importando pandas . . . 12
7.2 Lista de resultados . . . 12
7.3 Salvando linha de resultado . . . 12
7.4 Convertendo para DataFrame . . . 12
7.5 Exportando para CSV . . . 12

8 Comparação de resultados **13**
8.1 Carregando resultados . . . 13
8.2 Melhor acurácia geral . . . 13
8.3 Média por Classificador . . . 13
8.4 Média por Wavelet . . . 13
8.5 Média por Nível . . . 13

9 Brincadeiras (5 experimentos) **14**
9.1 Exp 1: GridSearchCV no Random Forest . . . 14
9.2 Exp 2: Kernel Polinomial SVM . . . 14
9.3 Exp 3: Wavelet db20 . . . 14
9.4 Exp 4: Combinando features (L4 + L5) . . . 14
9.5 Exp 5: Top features RF vs SVM . . . 14

10 Checklist **15**

11 Desafio final **16**

Capítulo 1

O que o plano exige

Neste mês, inicia-se a fase de expansão. O objetivo é expandir o espaço de configuração, aproveitando a capacidade de processamento (CPU). As atividades principais incluem:

- Adicionar os classificadores Support Vector Machine (SVM) e Random Forest (RF).
- Explorar novas *mother wavelets*, especificamente db8 e sym8.
- Aprofundar a decomposição para o nível $L = 5$.
- Comparar e registrar todas as combinações (corpus × nível × wavelet × classificador).

Marco / Entrega

Matriz de configuração expandida com resultados registrados sistematicamente.

Capítulo 2

SVM (Support Vector Machine)

O SVM busca encontrar um hiperplano que separe as classes com a maior margem possível. Utilizaremos o módulo `SVC` do `scikit-learn`.

Atenção

É essencial padronizar os dados antes de treinar um SVM. O `StandardScaler` é obrigatório!

2.1 Importando bibliotecas

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

2.2 Criando um Pipeline Básico

```
svm_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', C=1.0))
])
```

2.3 Treinamento Simples

```
# X_train, y_train pre-carregados
svm_pipeline.fit(X_train, y_train)
```

2.4 Predição e Acurácia

```
from sklearn.metrics import accuracy_score
y_pred_svm = svm_pipeline.predict(X_test)
```

```
acc = accuracy_score(y_test, y_pred_svm)
print(f"Acuracia SVM: {acc:.4f}")
```

2.5 GridSearchCV para Hiperparâmetros

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'svm__C': [0.1, 1, 10],
    'svm__kernel': ['linear', 'rbf']
}

grid = GridSearchCV(svm_pipeline, param_grid, cv=3)
```

2.6 Executando o Grid Search

```
grid.fit(X_train, y_train)
print("Melhores parametros:", grid.best_params_)
```

Capítulo 3

Random Forest

Random Forest é um ensemble de árvores de decisão. Não requer padronização dos dados e fornece a importância das features.

3.1 Importando Random Forest

```
from sklearn.ensemble import RandomForestClassifier
```

3.2 Instanciando o Modelo

```
rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
```

3.3 Treinamento RF

```
rf_clf.fit(X_train, y_train)
```

3.4 Avaliando RF

```
y_pred_rf = rf_clf.predict(X_test)
print(f"Acuracia RF: {accuracy_score(y_test, y_pred_rf):.4f}")
```

3.5 Importância das Features

```
importances = rf_clf.feature_importances_
print("Top 5 features:", importances.argsort()[-5:][::-1])
```

Capítulo 4

Novas Wavelets: db8 e sym8

As wavelets `db8` (Daubechies 8) e `sym8` (Symlets 8) são exploradas para capturar características diferentes.

4.1 Listando Wavelets

```
import pywt
print("Familia Daubechies:", pywt.wavelist('db'))
```

4.2 Propriedades db8

```
w_db8 = pywt.Wavelet('db8')
print("Tamanho do filtro db8:", w_db8.dec_len)
```

4.3 Propriedades sym8

```
w_sym8 = pywt.Wavelet('sym8')
print("Tamanho do filtro sym8:", w_sym8.dec_len)
```

4.4 Reconstrução com db8

```
coeffs_db8 = pywt.wavedec(audio, 'db8', level=4)
rec_db8 = pywt.waverec(coeffs_db8, 'db8')
```

4.5 Erro de reconstrução

```
import numpy as np
erro = np.max(np.abs(audio - rec_db8[:len(audio)]))
print("Erro db8:", erro)
```

Capítulo 5

Nível L=5

Expandindo para o nível $L = 5$, teremos $2^5 = 32$ sub-bandas se usarmos WPT completo, gerando 288 features (32×9 descritores).

5.1 WPT Nível 5

```
wp = pywt.WaveletPacket(data=audio, wavelet='db8', mode='symmetric'
    , maxlevel=5)
```

5.2 Listando nós Nível 5

```
nos_L5 = [node.path for node in wp.get_level(5, 'natural')]
print(f"Total de nos em L=5: {len(nos_L5)}")
```

5.3 Extraindo energia L=5

```
energias = [np.sum(wp[node].data ** 2) for node in nos_L5]
print("Primeiras 5 energias:", energias[:5])
```

5.4 Extração completa L=5

```
def extrair_features_L5(audio, wavelet_name):
    wp = pywt.WaveletPacket(data=audio, wavelet=wavelet_name,
        maxlevel=5)
    features = []
    for node in wp.get_level(5, 'natural'):
        d = wp[node].data
        features.extend([np.mean(d), np.var(d)]) # Exemplo
            simplificado
    return features
```

5.5 Teste L=5 vs L=4

```
f_l4 = extrair_features_L4(audio, 'db8')
f_l5 = extrair_features_L5(audio, 'db8')
print(f"Len L4: {len(f_l4)}, Len L5: {len(f_l5)}")
```

Capítulo 6

Matriz de configuração

Vamos utilizar `itertools.product` para iterar pelas 24 combinações (3 wavelets × 2 níveis × 4 classificadores).

6.1 Importando itertools

```
import itertools
```

6.2 Definindo opções

```
wavelets = ['db2', 'db8', 'sym8']
niveis = [4, 5]
classificadores = ['knn', 'lr', 'svm', 'rf']
```

6.3 Gerando combinações

```
combinacoes = list(itertools.product(wavelets, niveis,
    classificadores))
print(f"Total de combinacoes: {len(combinacoes)}")
```

6.4 Loop principal

```
for w, l, c in combinacoes:
    print(f"Executando: Wavelet={w}, Nivel={l}, Classif={c}")
```

6.5 Função get_classifier

```
def get_classifier(name):
    if name == 'svm': return Pipeline([('s', StandardScaler()), ('c
        ', SVC())])
    if name == 'rf': return RandomForestClassifier()
    # Adicionar KNN e LR...
    return None
```

Capítulo 7

Registrar experimentos

Precisamos salvar os resultados em CSV sistematicamente.

7.1 Importando pandas

```
import pandas as pd
```

7.2 Lista de resultados

```
resultados = []
```

7.3 Salvando linha de resultado

```
resultados.append({
    'Wavelet': 'db8',
    'Nivel': 5,
    'Classificador': 'svm',
    'Acuracia': 0.95
})
```

7.4 Convertendo para DataFrame

```
df_res = pd.DataFrame(resultados)
print(df_res.head())
```

7.5 Exportando para CSV

```
df_res.to_csv('resultados_mes7.csv', index=False)
```

Capítulo 8

Comparação de resultados

Vamos analisar os resultados agrupando o DataFrame.

8.1 Carregando resultados

```
df = pd.read_csv('resultados_mes7.csv')
```

8.2 Melhor acurácia geral

```
best = df.loc[df['Acuracia'].idxmax()]
print("Melhor conf:\n", best)
```

8.3 Média por Classificador

```
print(df.groupby('Classificador')['Acuracia'].mean())
```

8.4 Média por Wavelet

```
print(df.groupby('Wavelet')['Acuracia'].mean())
```

8.5 Média por Nível

```
print(df.groupby('Nivel')['Acuracia'].mean())
```

Capítulo 9

Brincadeiras (5 experimentos)

9.1 Exp 1: GridSearchCV no Random Forest

```
rf_params = {'n_estimators': [50, 100], 'max_depth': [None, 10]}
rf_grid = GridSearchCV(RandomForestClassifier(), rf_params)
```

9.2 Exp 2: Kernel Polinomial SVM

```
svm_poly = SVC(kernel='poly', degree=3)
# Treinar e avaliar
```

9.3 Exp 3: Wavelet db20

```
# db20 possui filtros bem longos, analisar distorcoes de borda
wp20 = pywt.WaveletPacket(data=audio, wavelet='db20', maxlevel=4)
```

9.4 Exp 4: Combinando features (L4 + L5)

```
features_comb = np.hstack([f_l4, f_l5])
```

9.5 Exp 5: Top features RF vs SVM

```
# Treinar RF, pegar indices das 10 melhores
idx_best = importances.argsort()[-10:]
# Treinar SVM apenas nessas 10 features
X_train_reduzido = X_train[:, idx_best]
```

Capítulo 10

Checklist

SVM treinado com StandardScaler

Random Forest incluído

db8 e sym8 testadas

Decomposição L=5 completa

Todos os 24 casos registrados em CSV

Capítulo 11

Desafio final

Desafio

Implemente um loop que salve uma matriz de confusão (imagem PNG) para as top 3 configurações encontradas na matriz de configuração.

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Exemplo de salvamento
# disp = ConfusionMatrixDisplay.from_estimator(...)
# plt.savefig('conf_matrix_top1.png')
```